

Four weaknesses. One clear takeaway.

Testing of a locally hosted OWASP Juice Shop instance identified four security findings across authorization, injection, client-side scripting, and information disclosure.

BOUNDARY

Controlled local assessment only

OWASP Juice Shop running on **localhost:3000**; evidence should not be read as a statement about a production system.

CORE TAKEAWAY

Four control layers need attention

Authentication, authorization, input handling and output encoding, plus web-server exposure controls.

ASSESSMENT FRAME

Scope, objectives, and methodology.

A controlled workflow for turning observed behavior into practical remediation.

SCOPE

The local OWASP Juice Shop training instance only.

OBJECTIVES

Understand weaknesses, perform controlled tests, capture evidence, assess impact, and document remediation.

METHODOLOGY

Identify functionality, test locally, observe responses, classify the weakness, assess impact, and record recommendations.

FINDINGS OVERVIEW

- **01** Product tampering · authorization
- **02** SQL injection · authentication
- **03** XSS · client-side execution
- **04** Sensitive data exposure · public files

Tools: web browser, Developer Tools, and Postman.

Four findings, one compact view.

Each finding is grounded in observed behavior from the local assessment.

ID	FINDING	OBSERVATION	IMPACT
01	Product tampering API authorization	PUT /api/Products/9 changed a product description and returned success.	Unauthorized users could alter application data and product trust.
02	SQL injection Authentication	Crafted SQL input in the email field was accepted without a valid password.	Login controls could be bypassed, exposing protected functionality.
03	Cross-site scripting Client-side injection	A harmless payload in search displayed a JavaScript alert.	Injected scripts could execute in a user's browser.
04	Sensitive data exposure Information disclosure	Public /ftp listing exposed confidential acquisitions.md.	Unauthorized users could access sensitive business information.

Prioritized remediation and conclusion.

1. **Protect authentication first:** eliminate SQL injection paths with parameterized queries or prepared statements, validate inputs, and verify login controls cannot be bypassed.
2. **Enforce authorization:** require authentication and permission checks on product update endpoints; limit changes to appropriately privileged users.
3. **Contain client-side execution:** apply context-appropriate output encoding and safe input handling, with a strong Content Security Policy where appropriate.
4. **Reduce information exposure:** remove sensitive files from web-accessible directories, apply access controls, and disable unnecessary directory listing.

Retest each local finding after remediation and retain the before-and-after evidence set.

The assessment demonstrated practical testing against the intentionally vulnerable OWASP Juice Shop, but the evidence is bounded to **localhost:3000** and is not a statement about a production system.